

```
// SPDX-License-Identifier: GPL-2.0
/*
 * virtio-gpu-nv: NVIDIA GPU ioctl proxy for libkrun VMs.
 *
 * Each guest open("/dev/nvidia*") creates a new host FD via the VMM.
 * Ioctls are forwarded over the control virtqueue; mmap requests result
 * in KVM memory slots set up by the VMM so hot-path GPU writes go direct
 * through EPT â\200\224 no VMM involvement in the render loop.
 *
 * Guest kernel driver â\200\224 runs inside the VM.
 * Place in: drivers/virtio/virtio_gpu_nv.c (libkrunfw tree)
 */
#include <drm/drm_drv.h>
#include <linux/cdev.h>
#include <linux/completion.h>
#include <linux/cpu.h>
#include <linux/file.h>
#include <linux/fs.h>
#include <linux/mm.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/proc_fs.h>
#include <linux/scatterlist.h>
#include <linux/seq_file.h>
#include <linux/slab.h>
#include <linux/uaccess.h>
#include <linux/virtio.h>
#include <linux/virtio_config.h>
#include <linux/virtio_ids.h>

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 virt
io device identity â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224
\200â\224\200 */

#define VIRTIO_ID_GPU_NV 45

/* Feature bits */
#define VIRTIO_GPU_NV_F_UVM 0
#define VIRTIO_GPU_NV_F_ENCODE 1
#define VIRTIO_GPU_NV_F_GRAPHICS 2

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 NVID
IA device node numbers â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â
\224\200â\224\200 */

#define NV_MAJOR 195
#define NV_CTL_MINOR 255

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 Wire
protocol constants â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224
\200â\224\200 */

#define NVGPU_MSG_OPEN 1
#define NVGPU_MSG_CLOSE 2
#define NVGPU_MSG_IOCTL 3
#define NVGPU_MSG_MMAP 4
#define NVGPU_MSG_MUNMAP 5
#define NVGPU_MSG_GET_PROC_FILES 6
#define NVGPU_MSG_GET_SYS_FILES 7

/* device_type values for OPEN */
#define NVGPU_DEV_CTL 255
#define NVGPU_DEV_UVM 256
#define NVGPU_DEV_UVM_TOOLS 257
#define NVGPU_DEV_MODESET 258

/* capability bits */
#define NVGPU_CAP_COMPUTE (1 << 0)
#define NVGPU_CAP_GRAPHICS (1 << 1)
#define NVGPU_CAP_VIDEO (1 << 2)
```

```
#define NVGPU_CAP_UTILITY (1 << 3)

/* NVIDIA ioctl numbers that require nested-pointer marshalling */
#define NV_ESC_RM_CONTROL 0x2a
#define NV_ESC_RM_ALLOC 0x2b
/* UVM_INITIALIZE ioctl nr */
#define UVM_INITIALIZE_NR 0x30

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 Wire
protocol structs â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200
\200â\224\200 */

struct nvgpu_msg_hdr {
    __le32 msg_type;
    __le32 handle;
    __le32 status;
    __le32 padding;
} __packed;

struct nvgpu_open_req {
    struct nvgpu_msg_hdr hdr;
    __le32 device_type;
    __le32 flags;
} __packed;

struct nvgpu_open_resp {
    struct nvgpu_msg_hdr hdr;
} __packed;

struct nvgpu_ioctl_req {
    struct nvgpu_msg_hdr hdr;
    __le32 cmd;
    __le32 data_len;
    __le32 nested_offset;
    __le32 nested_len;
    /* followed by: data_len bytes top-level struct,
     *                nested_len bytes nested data */
} __packed;

struct nvgpu_ioctl_resp {
    struct nvgpu_msg_hdr hdr;
    __le32 data_len;
    __le32 nested_len;
    /* followed by: data_len bytes modified top-level,
     *                nested_len bytes modified nested */
} __packed;

struct nvgpu_mmap_req {
    struct nvgpu_msg_hdr hdr;
    __le64 size;
    __le64 offset;
    __le32 prot;
    __le32 padding;
} __packed;

struct nvgpu_mmap_resp {
    struct nvgpu_msg_hdr hdr;
    __le64 guest_phys_addr;
    __le64 size;
    __le32 mapping_id;
    __le32 padding;
} __packed;

struct nvgpu_munmap_req {
    struct nvgpu_msg_hdr hdr;
    __le32 mapping_id;
    __le32 padding;
} __packed;

struct nvgpu_munmap_resp {
```

```

struct nvgpu_msg_hdr hdr;
} __packed;

/* VMM response: stream of nvgpu_proc_file_entry records,
 * terminated by an entry with path_len == 0 */
struct nvgpu_proc_file_entry {
    __le32 path_len; /* bytes in path[], 0 = end of stream */
    __le32 content_len; /* bytes in content[] */
    /* followed by: path_len bytes of path (no NUL),
     *                content_len bytes of content */
} __packed;

/* Per-GPU slot in VMM config space @\200\224 1088 bytes */
struct virtio_gpu_nv_gpu_slot {
    char pci_addr[16]; /* 0.. 16 directory name */
    __le32 minor; /* 16.. 20 /dev/nvidia<minor> */
    __le32 info_len; /* 20.. 24 valid bytes in info_text */
    __le32 padding[1]; /* 24.. 28 */
    char info_text[1060]; /* 28..1088 raw information content */
} __packed; /* 1088 bytes */

struct nvgpu_fd_translation_entry {
    __le32 nr;
    __le32 payload_offset;
} __packed;

/* VMM config space layout */
struct virtio_gpu_nv_config {
    char driver_version[32]; /* 0.. 32 */
    __le32 num_gpus; /* 32.. 36 */
    __le32 caps; /* 36.. 40 */
    __le32 gpu_device_ids[8]; /* 40.. 72 */
    struct virtio_gpu_nv_gpu_slot gpus[8]; /* 72.. */
    __le32 num_fd_translations;
    __le32 _pad;
    struct nvgpu_fd_translation_entry fd_translations[16];
} __packed;

static_assert(sizeof(struct virtio_gpu_nv_gpu_slot) == 1088,
               "gpu_slot size mismatch");
static_assert(sizeof(struct virtio_gpu_nv_config) == 8912,
               "virtio_gpu_nv_config size mismatch with VMM");
static_assert(offsetof(struct virtio_gpu_nv_config, num_fd_translations) ==
               8776,
               "fd_translations offset mismatch with VMM");

/* @\224\200@\224\200@\224\200@\224\200@\224\200@\224\200@\224\200 NVIDIA
IA ioctl parameter structs @\224\200@\224\200@\224\200@\224\200@\224\200@\224\200@\224\200@\224\200@ */

struct NVOS54_PARAMETERS {
    __le32 hClient;
    __le32 hObject;
    __le32 cmd;
    __le32 flags;
    __le64 params; /* pointer to sub-command data in guest VA */
    __le32 paramsSize;
    __le32 status;
} __packed;

struct NVOS64_PARAMETERS {
    __le32 hRoot;
    __le32 hObjectParent;
    __le32 hObjectNew;
    __le32 hClass;
    __le64 pAllocParams; /* pointer to class-specific alloc params */
    __le64 pRightsRequested; /* usually NULL */
    __le32 paramsSize;
    __le32 flags;
    __le32 status;

```



```
;
/* class for device_create() */
static struct class *nvgpu_class;

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 Virt
queue communication â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200
\200â\224\200 */

/*
 * nvgpu_send_recv â\200\224 submit one request to controlq and block until the VMM
 * returns the response. Caller must supply pre-allocated resp buffer.
 */
static int nvgpu_send_recv(struct nvgpu_device *dev, void *req, int req_len,
                          void *resp, int resp_len) {
    struct scatterlist sg_out, sg_in;
    struct scatterlist *sgs[2] = {&sg_out, &sg_in};
    int ret;

    mutex_lock(&dev->vq_lock);

    reinit_completion(&dev->req_done);
    dev->resp_buf = resp;
    dev->resp_len = resp_len;

    sg_init_one(&sg_out, req, req_len);
    sg_init_one(&sg_in, resp, resp_len);

    ret = virtqueue_add_sgs(dev->ctrl_vq, sgs, 1, 1, resp, GFP_KERNEL);
    if (ret < 0) {
        mutex_unlock(&dev->vq_lock);
        return ret;
    }

    mutex_unlock(&dev->vq_lock);
    virtqueue_kick(dev->ctrl_vq);

    ret = wait_for_completion_killable_timeout(&dev->req_done, 10 * HZ);
    if (ret == 0) {
        return -ETIMEDOUT;
    }
    if (ret < 0) {
        return ret;
    }

    return 0;
}

/* Virtqueue callback: VMM has written the response buffer */
static void nvgpu_ctrl_vq_cb(struct virtqueue *vq) {
    struct nvgpu_device *dev = vq->vdev->priv;
    void *buf;
    unsigned int len;

    while ((buf = virtqueue_get_buf(vq, &len)) != NULL) {
        complete(&dev->req_done);
    }
}

/* event virtqueue callback â\200\224 not used yet, just drain */
static void nvgpu_event_vq_cb(struct virtqueue *vq) {
    void *buf;
    unsigned int len;

    while ((buf = virtqueue_get_buf(vq, &len)) != NULL)
        /* TODO: deliver to waiting guest processes */;
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 Ioct
l forwarding â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â
```

```
\224\200 */
```

```
/*  
 * nvgpu_ioctl_simple â\200\224 flat struct, no embedded pointers.  
 * Covers ~95 % of NVIDIA ioctls.  
 */
```

```
static long nvgpu_ioctl_simple(struct nvgpu_fd *nfd, unsigned int cmd,  
                             void __user *uarg, unsigned int sz) {  
    int req_total = sizeof(struct nvgpu_ioctl_req) + sz;  
    int resp_max = sizeof(struct nvgpu_ioctl_resp) + sz;  
    void *req_buf, *resp_buf;  
    struct nvgpu_ioctl_req *req;  
    struct nvgpu_ioctl_resp *resp;  
    int ret;  
  
    req_buf = kmalloc(req_total, GFP_KERNEL);  
    resp_buf = kmalloc(resp_max, GFP_KERNEL);  
    if (!req_buf || !resp_buf) {  
        ret = -ENOMEM;  
        goto out;  
    }  
  
    req = (struct nvgpu_ioctl_req *)req_buf;  
    req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_IOCTL);  
    req->hdr.handle = cpu_to_le32(nfd->handle);  
    req->hdr.status = 0;  
    req->hdr.padding = 0;  
    req->cmd = cpu_to_le32(cmd);  
    req->data_len = cpu_to_le32(sz);  
    req->nested_offset = 0;  
    req->nested_len = 0;  
  
    if (copy_from_user(req_buf + sizeof(*req), uarg, sz)) {  
        ret = -EFAULT;  
        goto out;  
    }  
  
    ret = nvgpu_send_recv(nfd->dev, req_buf, req_total, resp_buf, resp_max);  
    if (ret < 0)  
        goto out;  
  
    resp = (struct nvgpu_ioctl_resp *)resp_buf;  
    ret = (int)le32_to_cpu((__le32)resp->hdr.status);  
  
    if (resp->data_len && le32_to_cpu(resp->data_len) <= sz) {  
        if (copy_to_user(uarg, resp_buf + sizeof(*resp),  
                        le32_to_cpu(resp->data_len)))  
            ret = -EFAULT;  
    }  
  
out:  
    kfree(req_buf);  
    kfree(resp_buf);  
    return ret;  
}
```

```
/*  
 * nvgpu_ioctl_rm_control â\200\224 NV_ESC_RM_CONTROL has a nested params buffer  
 * addressed via NVOS54_PARAMETERS.params.  
 */
```

```
static long nvgpu_ioctl_rm_control(struct nvgpu_fd *nfd, unsigned int cmd,  
                                  void __user *uarg, unsigned int sz) {  
    struct NVOS54_PARAMETERS params;  
    void __user *user_nested;  
    void *req_buf = NULL, *resp_buf = NULL, *nested;  
    struct nvgpu_ioctl_req *req;  
    struct nvgpu_ioctl_resp *resp;  
    int req_total, resp_max, ret;  
  
    if (sz < sizeof(params))
```

```
    return -EINVAL;

    if (copy_from_user(&params, uarg, sizeof(params)))
        return -EFAULT;

    user_nested = (void __user *) (unsigned long) le64_to_cpu(params.params);

    if (le32_to_cpu(params.paramsSize) > 1024 * 1024)
        return -EINVAL;

    req_total = sizeof(*req) + sizeof(params) + le32_to_cpu(params.paramsSize);
    resp_max = sizeof(struct nvgpu_ioctl_resp) + sizeof(params) +
        le32_to_cpu(params.paramsSize);

    req_buf = kmalloc(req_total, GFP_KERNEL);
    resp_buf = kmalloc(resp_max, GFP_KERNEL);
    if (!req_buf || !resp_buf) {
        ret = -ENOMEM;
        goto out;
    }

    req = (struct nvgpu_ioctl_req *) req_buf;
    req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_IOCTL);
    req->hdr.handle = cpu_to_le32(nfd->handle);
    req->hdr.status = 0;
    req->hdr.padding = 0;
    req->cmd = cpu_to_le32(cmd);
    req->data_len = cpu_to_le32(sizeof(params));
    req->nested_offset = cpu_to_le32(sizeof(params));
    req->nested_len = params.paramsSize; /* already LE */

    /* Copy top-level struct (guest pointer left in place; VMM ignores it) */
    memcpy(req_buf + sizeof(*req), &params, sizeof(params));

    /* Copy nested data from guest userspace */
    if (user_nested && le32_to_cpu(params.paramsSize) > 0) {
        nested = req_buf + sizeof(*req) + sizeof(params);
        if (copy_from_user(nested, user_nested, le32_to_cpu(params.paramsSize))) {
            ret = -EFAULT;
            goto out;
        }
    }

    ret = nvgpu_send_recv(nfd->dev, req_buf, req_total, resp_buf, resp_max);
    if (ret < 0)
        goto out;

    resp = (struct nvgpu_ioctl_resp *) resp_buf;
    ret = (int) (s32) le32_to_cpu((__le32) resp->hdr.status);

    /* Copy modified top-level struct back */
    if (copy_to_user(uarg, resp_buf + sizeof(*resp), sizeof(params))) {
        ret = -EFAULT;
        goto out;
    }

    /* Copy modified nested data back to original guest pointer */
    if (user_nested && le32_to_cpu(resp->nested_len) > 0) {
        if (copy_to_user(user_nested, resp_buf + sizeof(*resp) + sizeof(params),
            le32_to_cpu(resp->nested_len)))
            ret = -EFAULT;
    }

out:
    kfree(req_buf);
    kfree(resp_buf);
    return ret;
}

/*
```

```
* nvgpu_ioctl_rm_alloc â\200\224 NV_ESC_RM_ALLOC, same pattern via NVOS64_PARAMETERS.
*/
static long nvgpu_ioctl_rm_alloc(struct nvgpu_fd *nfd, unsigned int cmd,
                                void __user *uarg, unsigned int sz) {
    struct NVOS64_PARAMETERS params;
    void __user *user_alloc;
    void *req_buf = NULL, *resp_buf = NULL, *nested;
    struct nvgpu_ioctl_req *req;
    struct nvgpu_ioctl_resp *resp;
    int req_total, resp_max, ret;

    if (sz < sizeof(params))
        return -EINVAL;

    if (copy_from_user(&params, uarg, sizeof(params)))
        return -EFAULT;

    user_alloc = (void __user *) (unsigned long) le64_to_cpu(params.pAllocParms);

    if (le32_to_cpu(params.paramsSize) > 1024 * 1024)
        return -EINVAL;

    req_total = sizeof(*req) + sizeof(params) + le32_to_cpu(params.paramsSize);
    resp_max = sizeof(struct nvgpu_ioctl_resp) + sizeof(params) +
        le32_to_cpu(params.paramsSize);

    req_buf = kmalloc(req_total, GFP_KERNEL);
    resp_buf = kmalloc(resp_max, GFP_KERNEL);
    if (!req_buf || !resp_buf) {
        ret = -ENOMEM;
        goto out;
    }

    req = (struct nvgpu_ioctl_req *) req_buf;
    req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_IOCTL);
    req->hdr.handle = cpu_to_le32(nfd->handle);
    req->hdr.status = 0;
    req->hdr.padding = 0;
    req->cmd = cpu_to_le32(cmd);
    req->data_len = cpu_to_le32(sizeof(params));
    req->nested_offset = cpu_to_le32(sizeof(params));
    req->nested_len = params.paramsSize;

    memcpy(req_buf + sizeof(*req), &params, sizeof(params));

    if (user_alloc && le32_to_cpu(params.paramsSize) > 0) {
        nested = req_buf + sizeof(*req) + sizeof(params);
        if (copy_from_user(nested, user_alloc, le32_to_cpu(params.paramsSize))) {
            ret = -EFAULT;
            goto out;
        }
    }

    ret = nvgpu_send_recv(nfd->dev, req_buf, req_total, resp_buf, resp_max);
    if (ret < 0)
        goto out;

    resp = (struct nvgpu_ioctl_resp *) resp_buf;
    ret = (int) (s32) le32_to_cpu((__le32) resp->hdr.status);

    if (copy_to_user(uarg, resp_buf + sizeof(*resp), sizeof(params))) {
        ret = -EFAULT;
        goto out;
    }

    if (user_alloc && le32_to_cpu(resp->nested_len) > 0) {
        if (copy_to_user(user_alloc, resp_buf + sizeof(*resp) + sizeof(params),
            le32_to_cpu(resp->nested_len)))
            ret = -EFAULT;
    }
}
```

```
out:
    kfree(req_buf);
    kfree(resp_buf);
    return ret;
}

/*
 * NOTE: The only subtlety worth noting: copy_to_user on the way back writes the
 * VMM handle value (not a host fd number) back into the guest's buffer. That's
 * fine â\200\224 nvidia-smi doesn't read the payload back after REGISTER_FD, it only
 * checks the return code. If a future fd-carrying ioctl does need the response
 * payload, the VMM would need to translate back from host fd â\206\222 handle before
 * returning.
 */
static long nvgpu_ioctl_translate_fd(struct nvgpu_fd *nfd, unsigned int cmd,
                                     void __user *uarg, unsigned int sz,
                                     unsigned int payload_offset) {

    struct nvgpu_ioctl_req *req;
    struct nvgpu_ioctl_resp *resp;
    void *req_buf = NULL, *resp_buf = NULL;
    struct file *other_file;
    struct nvgpu_fd *other_nfd;
    int guest_fd;
    u32 host_handle;
    int req_total, resp_max, ret;

    if (sz < payload_offset + sizeof(guest_fd))
        return -EINVAL;

    req_total = sizeof(*req) + sz;
    resp_max = sizeof(*resp) + sz;

    req_buf = kmalloc(req_total, GFP_KERNEL);
    resp_buf = kmalloc(resp_max, GFP_KERNEL);
    if (!req_buf || !resp_buf) {
        ret = -ENOMEM;
        goto out;
    }

    /* Copy the full payload from userspace */
    if (copy_from_user(req_buf + sizeof(*req), uarg, sz)) {
        ret = -EFAULT;
        goto out;
    }

    /* Extract guest fd from its position in the payload */
    memcpy(&guest_fd, req_buf + sizeof(*req) + payload_offset, sizeof(guest_fd));

    /* Resolve guest fd â\206\222 nvgpu_fd â\206\222 VMM handle */
    other_file = fget(guest_fd);
    if (!other_file) {
        ret = -EBADF;
        goto out;
    }

    other_nfd = other_file->private_data;
    if (!other_nfd) {
        fput(other_file);
        ret = -EINVAL;
        goto out;
    }

    host_handle = other_nfd->handle;
    fput(other_file);

    /* Patch payload: replace raw guest fd with VMM handle */
    memcpy(req_buf + sizeof(*req) + payload_offset, &host_handle,
           sizeof(host_handle));
}
```

```
/* Build request header */
req = (struct nvgpu_ioctl_req *)req_buf;
req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_IOCTL);
req->hdr.handle = cpu_to_le32(nfd->handle);
req->hdr.status = 0;
req->hdr.padding = 0;
req->cmd = cpu_to_le32(cmd);
req->data_len = cpu_to_le32(sz);
req->nested_offset = 0;
req->nested_len = 0;

ret = nvgpu_send_recv(nfd->dev, req_buf, req_total, resp_buf, resp_max);
if (ret < 0)
    goto out;

resp = (struct nvgpu_ioctl_resp *)resp_buf;
ret = (int)(s32)le32_to_cpu((__le32)resp->hdr.status);

/* Write the (possibly modified) payload back to userspace */
if (ret == 0 && le32_to_cpu(resp->data_len) > 0) {
    if (copy_to_user(uarg, resp_buf + sizeof(*resp),
                    le32_to_cpu(resp->data_len)))
        ret = -EFAULT;
}

out:
    kfree(req_buf);
    kfree(resp_buf);
    return ret;
}

static const struct nvgpu_fd_translation_entry *
nvgpu_find_fd_translation(struct nvgpu_device *dev, unsigned int nr) {
    u32 i;
    for (i = 0; i < dev->num_fd_translations; i++)
        if (le32_to_cpu(dev->fd_translations[i].nr) == nr)
            return &dev->fd_translations[i];
    return NULL;
}

/* Main ioctl dispatcher */
static long nvgpu_ioctl(struct file *filp, unsigned int cmd,
                        unsigned long arg) {
    struct nvgpu_fd *nfd = filp->private_data;
    unsigned int nr = _IOC_NR(cmd);
    unsigned int sz = _IOC_SIZE(cmd);
    void __user *uarg = (void __user *)arg;
    const struct nvgpu_fd_translation_entry *fdt;

    /*
     * Most ioctls are small structs. NV_ESC_CARD_INFO is the exception:
     * its full form carries a table of up to 32 GPU entries and comes in
     * well above 4096 bytes on recent drivers (~5000 bytes for 535+).
     * Use a generous per-ioctl cap rather than one global limit.
     * Hard cap: no single ioctl struct should exceed 64 KiB
     */
    if (sz == 0 || sz > 65536)
        return -EINVAL;

    fdt = nvgpu_find_fd_translation(nfd->dev, nr);
    if (fdt)
        return nvgpu_ioctl_translate_fd(nfd, cmd, uarg, sz,
                                        le32_to_cpu(fdt->payload_offset));

    switch (nr) {
    case NV_ESC_RM_CONTROL:
        return nvgpu_ioctl_rm_control(nfd, cmd, uarg, sz);
    case NV_ESC_RM_ALLOC:
        return nvgpu_ioctl_rm_alloc(nfd, cmd, uarg, sz);
    default:

```

```

return nvgpu_ioctl_simple(nfd, cmd, uarg, sz);
}
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 UVM
ioctl â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 *
/

/*
 * UVM ioctls are all flat (no nested pointers), with one exception:
 * UVM_INITIALIZE gets the multi-process sharing flag injected.
 */
static long nvgpu_uvm_ioctl(struct file *filp, unsigned int cmd,
                           unsigned long arg) {
    struct nvgpu_fd *nfd = filp->private_data;
    unsigned int nr = _IOC_NR(cmd);
    unsigned int sz = _IOC_SIZE(cmd);
    void __user *uarg = (void __user *)arg;

    if (sz == 0)
        return -EINVAL;
    if (sz > 65536)
        return -EINVAL;

    if (nr == UVM_INITIALIZE_NR) {
        /*
         * Set UVM_INIT_FLAGS_MULTI_PROCESS_SHARING_MODE so the host
         * driver allows this UVM context to be shared across processes
         * (one per VM). The flag is at offset 0 in UvmInitializeParams.
         * We read, OR the bit, write back, then forward.
         */
        u64 flags;

        if (sz >= sizeof(flags)) {
            if (copy_from_user(&flags, uarg, sizeof(flags)))
                return -EFAULT;
            flags |= (1ULL << 2); /* UVM_INIT_FLAGS_MULTI_PROCESS_SHARING_MODE */
            if (copy_to_user(uarg, &flags, sizeof(flags)))
                return -EFAULT;
        }
    }

    return nvgpu_ioctl_simple(nfd, cmd, uarg, sz);
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 mmap
â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 */

static void nvgpu_vma_close(struct vm_area_struct *vma) {
    struct nvgpu_fd *nfd = vma->vm_file->private_data;
    u32 mapping_id = (u32)(unsigned long)vma->vm_private_data;
    struct nvgpu_munmap_req *req;
    struct nvgpu_munmap_resp *resp;

    req = kzalloc(sizeof(*req), GFP_KERNEL);
    resp = kzalloc(sizeof(*resp), GFP_KERNEL);
    if (req && resp) {
        req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_MUNMAP);
        req->hdr.handle = cpu_to_le32(nfd->handle);
        req->mapping_id = cpu_to_le32(mapping_id);
        nvgpu_send_recv(nfd->dev, req, sizeof(*req), resp, sizeof(*resp));
    }
    kfree(req);
    kfree(resp);
}

static const struct vm_operations_struct nvgpu_vm_ops = {
    .close = nvgpu_vma_close,
};
```

```
static int nvgpu_mmap(struct file *filp, struct vm_area_struct *vma) {
    struct nvgpu_fd *nfd = filp->private_data;
    u64 size = vma->vm_end - vma->vm_start;
    u64 offset = (u64)vma->vm_pgoff << PAGE_SHIFT;
    struct nvgpu_mmap_req *req;
    struct nvgpu_mmap_resp *resp;
    int ret;

    req = kzalloc(sizeof(*req), GFP_KERNEL);
    resp = kzalloc(sizeof(*resp), GFP_KERNEL);
    if (!req || !resp) {
        ret = -ENOMEM;
        goto out;
    }

    req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_MMAP);
    req->hdr.handle = cpu_to_le32(nfd->handle);
    req->size = cpu_to_le64(size);
    req->offset = cpu_to_le64(offset);
    req->prot = cpu_to_le32((vma->vm_flags & VM_WRITE) ? 3 : 1);

    ret = nvgpu_send_recv(nfd->dev, req, sizeof(*req), resp, sizeof(*resp));
    if (ret < 0)
        goto out;

    if ((s32)le32_to_cpu((__le32)resp->hdr.status) < 0) {
        ret = (s32)le32_to_cpu((__le32)resp->hdr.status);
        goto out;
    }

    vm_flags_set(vma, VM_IO | VM_PFNMAP | VM_DONTEXPAND | VM_DONTDUMP);
    vma->vm_page_prot = pgprot_writecombine(vma->vm_page_prot);

    ret = remap_pfn_range(vma, vma->vm_start,
                          le64_to_cpu(resp->guest_phys_addr) >> PAGE_SHIFT, size,
                          vma->vm_page_prot);
    if (ret)
        goto out;

    vma->vm_ops = &nvgpu_vm_ops;
    vma->vm_private_data = (void *) (unsigned long) le32_to_cpu(resp->mapping_id);

out:
    kfree(req);
    kfree(resp);
    return ret;
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 open
 / release â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200
 \200 */

static int nvgpu_open_common(struct inode *inode, struct file *filp,
                             u32 device_type) {
    struct nvgpu_device *dev;
    struct nvgpu_fd *nfd;
    struct nvgpu_open_req *req;
    struct nvgpu_open_resp *resp;
    int ret;

    /* Recover nvgpu_device pointer depending on which cdev was opened */
    if (device_type == NVGPU_DEV_UVM || device_type == NVGPU_DEV_UVM_TOOLS)
        dev = container_of(inode->i_cdev, struct nvgpu_device, cdev_uvm);
    else if (device_type == NVGPU_DEV_CTL)
        dev = container_of(inode->i_cdev, struct nvgpu_device, cdev_ctl);
    else
        dev = container_of(inode->i_cdev, struct nvgpu_device,
                           cdev_gpu[iminor(inode)]);

    nfd = kzalloc(sizeof(*nfd), GFP_KERNEL);
```

```
req = kzalloc(sizeof(*req), GFP_KERNEL);
resp = kzalloc(sizeof(*resp), GFP_KERNEL);
if (!nfd || !req || !resp) {
    kfree(nfd);
    kfree(req);
    kfree(resp);
    return -ENOMEM;
}

nfd->dev = dev;
nfd->device_type = device_type;

req->hdr.msg_type = cpu_to_le32(NVGPU_MSG_OPEN);
req->device_type = cpu_to_le32(device_type);
req->flags = cpu_to_le32(filp->f_flags);

ret = nvgpu_send_recv(dev, req, sizeof(*req), resp, sizeof(*resp));
if (ret < 0 || (s32)le32_to_cpu((__le32)resp->hdr.status) < 0) {
    kfree(nfd);
    kfree(req);
    kfree(resp);
    if (ret < 0)
        return ret;
    return (s32)le32_to_cpu((__le32)resp->hdr.status);
}

nfd->handle = le32_to_cpu(resp->hdr.handle);
filp->private_data = nfd;
kfree(req);
kfree(resp);
return 0;
}

static int nvgpu_gpu_open(struct inode *inode, struct file *filp) {
    return nvgpu_open_common(inode, filp, (u32)iminor(inode));
}

static int nvgpu_ctl_open(struct inode *inode, struct file *filp) {
    return nvgpu_open_common(inode, filp, NVGPU_DEV_CTL);
}

static int nvgpu_uvm_open(struct inode *inode, struct file *filp) {
    return nvgpu_open_common(inode, filp, NVGPU_DEV_UVM);
}

static int nvgpu_release(struct inode *inode, struct file *filp) {
    struct nvgpu_fd *nfd = filp->private_data;
    struct nvgpu_msg_hdr *req;
    struct nvgpu_msg_hdr *resp;

    req = kzalloc(sizeof(*req), GFP_KERNEL);
    resp = kzalloc(sizeof(*resp), GFP_KERNEL);
    if (req && resp) {
        req->msg_type = cpu_to_le32(NVGPU_MSG_CLOSE);
        req->handle = cpu_to_le32(nfd->handle);
        nvgpu_send_recv(nfd->dev, req, sizeof(*req), resp, sizeof(*resp));
    }
    kfree(req);
    kfree(resp);
    kfree(nfd);
    return 0;
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 file
_operations tables â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200
\200â\224\200 */

static const struct file_operations nvgpu_gpu_fops = {
    .owner = THIS_MODULE,
    .open = nvgpu_gpu_open,
```

```
.release = nvgpu_release,  
.unlocked_ioctl = nvgpu_ioctl,  
.mmap = nvgpu_mmap,  
};  
  
static const struct file_operations nvgpu_ctl_fops = {  
.owner = THIS_MODULE,  
.open = nvgpu_ctl_open,  
.release = nvgpu_release,  
.unlocked_ioctl = nvgpu_ioctl,  
.mmap = nvgpu_mmap,  
};  
  
static const struct file_operations nvgpu_uvm_fops = {  
.owner = THIS_MODULE,  
.open = nvgpu_uvm_open,  
.release = nvgpu_release,  
.unlocked_ioctl = nvgpu_uvm_ioctl,  
.mmap = nvgpu_mmap,  
};  
  
/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 /pro  
c/driver/nvidia â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200  
\224\200 */  
  
static int nvgpu_proc_version_show(struct seq_file *m, void *v) {  
    struct nvgpu_device *dev = m->private;  
  
    seq_printf(m,  
        "NVRM version: NVIDIA UNIX x86_64 Kernel Module  %s\n"  
        "GCC version: gcc version 12.2.0\n",  
        dev->driver_version);  
    return 0;  
}  
  
static int nvgpu_proc_version_open(struct inode *inode, struct file *filp) {  
    return single_open(filp, nvgpu_proc_version_show, pde_data(inode));  
}  
  
static const struct proc_ops nvgpu_proc_version_ops = {  
.proc_open = nvgpu_proc_version_open,  
.proc_read = seq_read,  
.proc_lseek = seq_lseek,  
.proc_release = single_release,  
};  
  
static int nvgpu_proc_params_show(struct seq_file *m, void *v) {  
    struct nvgpu_device *dev = m->private;  
  
    seq_printf(m, "NVreg_EnablePCIeGen3=1\n"  
        "NVreg_MemoryPoolSize=0\n");  
    (void)dev;  
    return 0;  
}  
  
static int nvgpu_proc_params_open(struct inode *inode, struct file *filp) {  
    return single_open(filp, nvgpu_proc_params_show, pde_data(inode));  
}  
  
static const struct proc_ops nvgpu_proc_params_ops = {  
.proc_open = nvgpu_proc_params_open,  
.proc_read = seq_read,  
.proc_lseek = seq_lseek,  
.proc_release = single_release,  
};  
  
/* Generic heap-backed proc file â\200\224 used for all passthrough files */  
struct nvgpu_proc_buf {  
    char *data;  
    size_t len;
```

```
};

static int nvgpu_proc_buf_show(struct seq_file *m, void *v) {
    struct nvgpu_proc_buf *b = m->private;
    seq_write(m, b->data, b->len);
    return 0;
}

static int nvgpu_proc_buf_open(struct inode *inode, struct file *filp) {
    return single_open(filp, nvgpu_proc_buf_show, pde_data(inode));
}

static const struct proc_ops nvgpu_proc_buf_ops = {
    .proc_open = nvgpu_proc_buf_open,
    .proc_read = seq_read,
    .proc_lseek = seq_lseek,
    .proc_release = single_release,
};

/* Simple directory cache â\200\224 avoids duplicate proc_mkdir calls */
#define NVGPU_PROC_MAX_DIRS 32

struct nvgpu_proc_dir_cache {
    char path[128];
    struct proc_dir_entry *entry;
};

static struct nvgpu_proc_dir_cache nvgpu_dir_cache[NVGPU_PROC_MAX_DIRS];
static int nvgpu_dir_cache_count;

static void nvgpu_dir_cache_reset(void) {
    memset(nvgpu_dir_cache, 0, sizeof(nvgpu_dir_cache));
    nvgpu_dir_cache_count = 0;
}

static struct proc_dir_entry *static struct proc_dir_entry *
nvgpu_proc_mkdir_cached(const char *path, struct proc_dir_entry *parent) {
    int i;
    struct proc_dir_entry *entry;

    /* Check our cache first */
    for (i = 0; i < nvgpu_dir_cache_count; i++)
        if (strcmp(nvgpu_dir_cache[i].path, path) == 0)
            return nvgpu_dir_cache[i].entry;

    /*
     * "driver" already exists in procfs â\200\224 don't try to recreate it.
     * Add other known-existing dirs here if the VMM sends more paths
     * that collide with built-in proc entries.
     */
    if (parent == NULL && strcmp(path, "driver") == 0)
        entry = NULL; /* proc_create_data with NULL parent+leaf works */
    else
        entry = proc_mkdir(path, parent);

    if (nvgpu_dir_cache_count < NVGPU_PROC_MAX_DIRS) {
        strncpy(nvgpu_dir_cache[nvgpu_dir_cache_count].path, path, 128);
        nvgpu_dir_cache[nvgpu_dir_cache_count].entry = entry;
        nvgpu_dir_cache_count++;
    }

    return entry;
}

static struct proc_dir_entry *nvgpu_proc_mkdir_parents(char *pathbuf,
                                                       char **leaf_name) {
    struct proc_dir_entry *parent = NULL;
    char built[256] = {};
    char *slash #include<drm / drm_drv.h>;
    char *p;
```

```
slash = strrchr(pathbuf, '/');
if (!slash) {
    *leaf_name = pathbuf;
    return NULL;
}

*leaf_name = slash + 1;
*slash = '\\0';

/* Walk each component, building the full path as we go
 * so the cache key is always the full absolute component */
p = pathbuf;
while (*p) {
    char *next = strchr(p, '/');
    if (next)
        *next = '\\0';

    /* Append component to built path */
    if (built[0])
        strlcat(built, "/", sizeof(built));
    strlcat(built, p, sizeof(built));

    parent = nvgpu_proc_mkdir_cached(built, NULL);

    if (next) {
        *next = '/';
        p = next + 1;
    } else {
        break;
    }
}

return parent;
}

static int nvgpu_proc_init(struct nvgpu_device *dev) {
    struct nvgpu_msg_hdr *req;
    u8 *resp_buf, *p, *end;
    /* 512 KiB â200â224 vastly more than needed, avoids any size guessing */
    const size_t resp_size = 512 * 1024;
    int ret = 0;

    req = kzalloc(sizeof(*req), GFP_KERNEL);
    if (!req)
        return -ENOMEM;

    resp_buf = kvmalloc(resp_size, GFP_KERNEL);
    if (!resp_buf) {
        kfree(req);
        return -ENOMEM;
    }

    req->msg_type = cpu_to_le32(NVGPU_MSG_GET_PROC_FILES);
    req->handle = 0;
    req->status = 0;
    req->padding = 0;

    ret = nvgpu_send_recv(dev, req, sizeof(*req), resp_buf, resp_size);
    if (ret < 0) {
        dev_err(&dev->vdev->dev, "virtio-gpu-nv: GET_PROC_FILES failed: %d\\n", ret);
        goto out;
    }

    p = resp_buf;
    end = resp_buf + resp_size;

    nvgpu_dir_cache_reset();

    while (p + 8 <= end) {
```

```

    u32 path_len, content_len;
    struct nvgpu_proc_buf *buf;
    char *pathbuf, *leaf;
    struct proc_dir_entry *parent = NULL;

    memcpy(&path_len, p, 4);
    path_len = le32_to_cpu((__le32)path_len);
    memcpy(&content_len, p + 4, 4);
    content_len = le32_to_cpu((__le32)content_len);
    p += 8;

    if (path_len == 0)
        break; /* terminator */

    if (p + path_len + content_len > end) {
        dev_warn(&dev->vdev->dev, "virtio-gpu-nv: proc stream truncated\n");
        break;
    }

    buf = kzalloc(sizeof(*buf), GFP_KERNEL);
    if (!buf) {
        ret = -ENOMEM;
        goto out;
    }

    buf->data = kmemdup(p + path_len, content_len, GFP_KERNEL);
    if (!buf->data) {
        kfree(buf);
        ret = -ENOMEM;
        goto out;
    }
    buf->len = content_len;

    pathbuf = kmalloc(path_len + 1, GFP_KERNEL);
    if (!pathbuf) {
        kfree(buf->data);
        kfree(buf);
        ret = -ENOMEM;
        goto out;
    }
    memcpy(pathbuf, p, path_len);
    pathbuf[path_len] = '\0';

    parent = nvgpu_proc_mkdir_parents(pathbuf, &leaf);
    proc_create_data(leaf, 0444, parent, &nvgpu_proc_buf_ops, buf);
    dev_dbg(&dev->vdev->dev, "virtio-gpu-nv: /proc/%s (%u bytes)\n", pathbuf,
            content_len);

    kfree(pathbuf);
    p += path_len + content_len;
}

out:
    kvfree(resp_buf);
    kfree(req);
    return ret;
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 Modu
le-global device pointer â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â
\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200
â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224
\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200
* libkrun is single-VM-per-process so there is exactly one nvgpu_device.
* sysfs show callbacks need access to the stored host content; they use
* this pointer rather than trying to recover it from the kobject tree.
* Set in nvgpu_probe, cleared in nvgpu_remove.
*/
static struct nvgpu_device *g_nvgpu_dev;
```



```

return 0; /* non-fatal */
}

cpu0 = get_cpu_device(0);
if (!cpu0 || !cpu0->kobj.parent || !cpu0->kobj.parent->parent) {
    dev_warn(&dev->vdev->dev,
        "virtio-gpu-nv: unexpected cpu0 kobj hierarchy\n");
    return 0;
}
system_kobj = cpu0->kobj.parent->parent;

/* Rename attributes to the canonical sysfs names */
nvgpu_attr_cpumap.attr.name = "cpumap";
nvgpu_attr_cpulist.attr.name = "cpulist";
nvgpu_attr_numastat.attr.name = "numastat";
nvgpu_attr_meminfo.attr.name = "meminfo";

dev->numa_node_kobj = kobject_create_and_add("node", system_kobj);
if (!dev->numa_node_kobj) {
    dev_info(&dev->vdev->dev, "virtio-gpu-nv: /sys/.../node already exists\n");
    return 0; /* kernel has NUMA built-in â\200\224 fine */
}

dev->numa_node0_kobj = kobject_create_and_add("node0", dev->numa_node_kobj);
if (!dev->numa_node0_kobj) {
    dev_err(&dev->vdev->dev, "virtio-gpu-nv: node0 kobject_create failed\n");
    ret = -ENOMEM;
    goto err_node;
}

ret = sysfs_create_group(dev->numa_node0_kobj, &nvgpu_node0_group);
if (ret) {
    dev_err(&dev->vdev->dev, "virtio-gpu-nv: node0 sysfs_create_group: %d\n",
        ret);
    goto err_node0;
}

dev_info(&dev->vdev->dev,
    "virtio-gpu-nv: created /sys/devices/system/node/node0 "
    "(cpumap=%s)\n",
    dev->numa_cpumap);
return 0;

err_node0:
kobject_put(dev->numa_node0_kobj);
dev->numa_node0_kobj = NULL;
err_node:
kobject_put(dev->numa_node_kobj);
dev->numa_node_kobj = NULL;
return ret;
}

static void nvgpu_numa_cleanup(struct nvgpu_device *dev) {
    if (dev->numa_node0_kobj) {
        sysfs_remove_group(dev->numa_node0_kobj, &nvgpu_node0_group);
        kobject_put(dev->numa_node0_kobj);
        dev->numa_node0_kobj = NULL;
    }
    if (dev->numa_node_kobj) {
        kobject_put(dev->numa_node_kobj);
        dev->numa_node_kobj = NULL;
    }
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 DRI
device nodes (host major:minor passthrough) â\224\200â\224\200â\224\200â\224\200
â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224
\200 */
/*
 * The host has /dev/dri/card1 and /dev/dri/renderD128.

```

```
* The VMM finds which ones belong to the passed-through GPU (via sysfs
* symlinks) and reports their real major:minor in the GET_SYS_FILES response.
* We register character devices with those exact numbers so the guest sees
* the same /dev/dri/ layout.
*/

static struct class *nvgpu_drm_class;

static int nvgpu_dri_open(struct inode *inode, struct file *filp) {
    /* Delegate to GPU 0's open path */
    return nvgpu_open_common(inode, filp, 0);
}

static const struct file_operations nvgpu_dri_fops = {
    .owner = THIS_MODULE,
    .open = nvgpu_dri_open,
    .release = nvgpu_release,
    .unlocked_ioctl = nvgpu_ioctl,
    .mmap = nvgpu_mmap,
};

static char *nvgpu_devnode(const struct device *dev, umode_t *mode) {
    if (mode)
        *mode = 0666;
    return NULL;
}

static int nvgpu_dri_init(struct nvgpu_device *dev) {
    int i;
    struct class *drm_cls;

    if (dev->num_dri_devs == 0)
        return 0;

    /*
     * drm_virtio never ran (no standard virtio-gpu device exposed by VMM),
     * so /sys/class/drm was never created. We need it for udev to create
     * /dev/dri/ nodes. Create it here if absent; if DRM core already made
     * it, class_create returns ERR_PTR(-EEXIST) and we just move on without
     * owning it.
     */
    drm_cls = class_create("drm");
    if (IS_ERR(drm_cls)) {
        /* Already exists â\200\224 DRM core owns it, we don't */
        nvgpu_drm_class = NULL;
        dev_info(&dev->vdev->dev, "virtio-gpu-nv: drm class already exists\n");
    } else {
        /* We created it â\200\224 we own it, must destroy on remove */
        nvgpu_drm_class = drm_cls;
        nvgpu_drm_class->devnode = nvgpu_devnode;
        dev_info(&dev->vdev->dev, "virtio-gpu-nv: created drm class\n");
    }

    for (i = 0; i < dev->num_dri_devs; i++) {
        dev_t devno = MKDEV(dev->dri_devs[i].major, dev->dri_devs[i].minor);

        if (register_chrdev_region(devno, 1, dev->dri_devs[i].name) != 0) {
            dev_warn(&dev->vdev->dev, "virtio-gpu-nv: cannot reserve %s (%u:%u)\n",
                    dev->dri_devs[i].name, dev->dri_devs[i].major,
                    dev->dri_devs[i].minor);
            continue;
        }

        cdev_init(&dev->dri_devs[i].cdev, &nvgpu_dri_fops);
        dev->dri_devs[i].cdev.owner = THIS_MODULE;

        if (cdev_add(&dev->dri_devs[i].cdev, devno, 1) != 0) {
            unregister_chrdev_region(devno, 1);
            dev_warn(&dev->vdev->dev, "virtio-gpu-nv: cdev_add %s failed\n",
                    dev->dri_devs[i].name);
        }
    }
}
```

[illegible]

```

struct nvgpu_msg_hdr *req;
u8 *resp_buf;
u8 *p, *end;
int resp_max = 128 * 1024; /* 128 KiB plenty for a few sysfs files */
int ret = 0;

req = kzalloc(sizeof(*req), GFP_KERNEL);
if (!req)
    return -ENOMEM;

resp_buf = kvmalloc(resp_max, GFP_KERNEL);
if (!resp_buf) {
    kfree(req);
    return -ENOMEM;
}

req->msg_type = cpu_to_le32(NVGPU_MSG_GET_SYS_FILES);

ret = nvgpu_send_recv(dev, req, sizeof(*req), resp_buf, resp_max);
if (ret < 0)
    goto out;

p = resp_buf;
end = resp_buf + resp_max;

/* Section 1: sysfs files */
while (p + 8 <= end) {
    u32 path_len = le32_to_cpu(*(u32 *)p);
    p += 4;
    u32 content_len = le32_to_cpu(*(u32 *)p);
    p += 4;

    if (path_len == 0 && content_len == 0)
        break; /* terminator */

    if (p + path_len + content_len > end)
        break;

    /* NUL-safe path extraction */
    char path[256] = {};
    u32 copy_len = min(path_len, (u32)(sizeof(path) - 1));
    memcpy(path, p, copy_len);
    p += path_len;

    /*
     * Route by path suffix. VMM sends paths relative to /sys/,
     * e.g. "devices/system/node/node0/cpumap".
     */
    if (strstr(path, "node0/cpumap")) {
        u32 l = min(content_len, (u32)sizeof(dev->numa_cpumap) - 1);
        memcpy(dev->numa_cpumap, p, l);
        dev->numa_cpumap[l] = '\0';
    } else if (strstr(path, "node0/cpulist")) {
        u32 l = min(content_len, (u32)sizeof(dev->numa_cpulist) - 1);
        memcpy(dev->numa_cpulist, p, l);
        dev->numa_cpulist[l] = '\0';
    } else if (strstr(path, "node0/numastat")) {
        u32 l = min(content_len, (u32)sizeof(dev->numa_numastat) - 1);
        memcpy(dev->numa_numastat, p, l);
        dev->numa_numastat[l] = '\0';
    } else if (strstr(path, "node0/meminfo")) {
        u32 l = min(content_len, (u32)sizeof(dev->numa_meminfo) - 1);
        memcpy(dev->numa_meminfo, p, l);
        dev->numa_meminfo[l] = '\0';
    }
}
/* Unknown paths are silently ignored */

```

```

    p += content_len;
}

/* Section 2: DRI devices */
/* Remove */
if (p + 4 > end)
    goto out;

{
    u32 num_dri = le32_to_cpu(*(u32 *)p);
    p += 4;

    num_dri = min(num_dri, (u32)NVGPU_MAX_DRI_DEVS);
    dev->num_dri_devs = 0;

    for (u32 i = 0; i < num_dri && p + 12 <= end; i++) {
        u32 name_len = le32_to_cpu(*(u32 *)p);
        p += 4;
        u32 major = le32_to_cpu(*(u32 *)p);
        p += 4;
        u32 minor = le32_to_cpu(*(u32 *)p);
        p += 4;

        if (name_len == 0 || p + name_len > end)
            break;

        int idx = dev->num_dri_devs;
        u32 nl = min(name_len, (u32)(sizeof(dev->dri_devs[idx].name) - 1));
        memcpy(dev->dri_devs[idx].name, p, nl);
        dev->dri_devs[idx].name[nl] = '\0';
        dev->dri_devs[idx].major = major;
        dev->dri_devs[idx].minor = minor;
        dev->num_dri_devs++;

        dev_info(&dev->vdev->dev, "virtio-gpu-nv: DRI %s (%u:%u) from host\n",
                 dev->dri_devs[idx].name, major, minor);
        p += name_len;
    }
}

out:
    kvfree(resp_buf);
    kfree(req);
    return ret;
}

/* Prob
e / remove */

static int nvgpu_probe(struct virtio_device *vdev) {
    struct nvgpu_device *dev;
    struct virtqueue_info vqs_info[] = {
        {"control", nvgpu_ctrl_vq_cb},
        {"event", nvgpu_event_vq_cb},
    };
    struct virtqueue *vqs[2];
    dev_t gpu_devno;
    int ret, i;

    dev = devm_kzalloc(&vdev->dev, sizeof(*dev), GFP_KERNEL);
    if (!dev)
        return -ENOMEM;

    dev->vdev = vdev;
    vdev->priv = dev;

```

```
mutex_init(&dev->vq_lock);
init_completion(&dev->req_done);

/* Find virtqueues */
ret = virtio_find_vqs(vdev, 2, vqs, vqs_info, NULL);
if (ret)
    return ret;

dev->ctrl_vq = vqs[0];
dev->event_vq = vqs[1];

/* Read config space written by the VMM at device creation */
virtio_cread_bytes(vdev, 0, dev->driver_version, 32);
dev->driver_version[31] = '\0';
virtio_cread(vdev, struct virtio_gpu_nv_config, num_gpus, &dev->num_gpus);
virtio_cread(vdev, struct virtio_gpu_nv_config, caps, &dev->caps);

if (dev->num_gpus == 0 || dev->num_gpus > 248) {
    dev_err(&vdev->dev, "virtio-gpu-nv: bad num_gpus %u\n", dev->num_gpus);
    return -EINVAL;
}

/* GPU info records */
{
    u32 i;
    for (i = 0; i < dev->num_gpus && i < 8; i++) {
        size_t off = offsetof(struct virtio_gpu_nv_config, gpus[i]);
        virtio_cread_bytes(vdev, off, &dev->gpu_slots[i],
                           sizeof(dev->gpu_slots[i]));

        /* Safety: ensure pci_addr is NUL-terminated before logging */
        dev->gpu_slots[i].pci_addr[15] = '\0';

        dev_info(&vdev->dev,
                 "virtio-gpu-nv: GPU%u pci=%s minor=%u info_len=%u\n", i,
                 dev->gpu_slots[i].pci_addr, le32_to_cpu(dev->gpu_slots[i].minor),
                 le32_to_cpu(dev->gpu_slots[i].info_len));
    }
}

/* FD translation table */
virtio_cread(vdev, struct virtio_gpu_nv_config, num_fd_translations,
             &dev->num_fd_translations);

if (dev->num_fd_translations > 16)
    dev->num_fd_translations = 16;

if (dev->num_fd_translations > 0) {
    size_t off = offsetof(struct virtio_gpu_nv_config, fd_translations);
    virtio_cread_bytes(vdev, off, dev->fd_translations,
                       dev->num_fd_translations *
                       sizeof(dev->fd_translations[0]));
}

dev_info(&vdev->dev, "virtio-gpu-nv: %u fd-translation ioctl(s) registered\n",
        dev->num_fd_translations);

/* Ensure virtio is running before we open devices */
virtio_device_ready(vdev);

/* Create device class once */
nvgpu_class = class_create("nvidia");
if (IS_ERR(nvgpu_class)) {
    ret = PTR_ERR(nvgpu_class);
    nvgpu_class = NULL;
    return ret;
}

/* Set more open permissions to device node */
nvgpu_class->devnode = nvgpu_devnode;
```

```
/* Register /dev/nvidia0 â\200| /dev/nvidia<N-1> */
gpu_devno = MKDEV(NV_MAJOR, 0);
ret = register_chrdev_region(gpu_devno, dev->num_gpus, "nvidia");
if (ret)
    goto err_class;

for (i = 0; i < (int)dev->num_gpus; i++) {
    cdev_init(&dev->cdev_gpu[i], &nvgpu_gpu_fops);
    dev->cdev_gpu[i].owner = THIS_MODULE;
    ret = cdev_add(&dev->cdev_gpu[i], MKDEV(NV_MAJOR, i), 1);
    if (ret)
        goto err_gpu_cdevs;
    device_create(nvgpu_class, &vdev->dev, MKDEV(NV_MAJOR, i), NULL, "nvidia%d",
        i);
}

/* Register /dev/nvidiactl */
ret = register_chrdev_region(MKDEV(NV_MAJOR, NV_CTL_MINOR), 1, "nvidiactl");
if (ret)
    goto err_gpu_cdevs;

cdev_init(&dev->cdev_ctl, &nvgpu_ctl_fops);
dev->cdev_ctl.owner = THIS_MODULE;
ret = cdev_add(&dev->cdev_ctl, MKDEV(NV_MAJOR, NV_CTL_MINOR), 1);
if (ret)
    goto err_ctl_region;
device_create(nvgpu_class, &vdev->dev, MKDEV(NV_MAJOR, NV_CTL_MINOR), NULL,
    "nvidiactl");

/* Register /dev/nvidia-uvm (dynamic major) */
ret = alloc_chrdev_region(&dev->uvm_devno, 0, 2, "nvidia-uvm");
if (ret)
    goto err_ctl_cdev;

cdev_init(&dev->cdev_uvm, &nvgpu_uvm_fops);
dev->cdev_uvm.owner = THIS_MODULE;
ret = cdev_add(&dev->cdev_uvm, dev->uvm_devno, 1);
if (ret)
    goto err_uvm_region;

device_create(nvgpu_class, &vdev->dev, dev->uvm_devno, NULL, "nvidia-uvm");
device_create(nvgpu_class, &vdev->dev, MKDEV(MAJOR(dev->uvm_devno), 1), NULL,
    "nvidia-uvm-tools");

/* Create /proc/driver/nvidia/version */
ret = nvgpu_proc_init(dev);
if (ret)
    goto err_uvm_cdev;

/* Set global pointer before creating sysfs/DRI (show callbacks need it) */
g_nvgpu_dev = dev;

/* Fetch host sysfs content + DRI device list from the VMM */
ret = nvgpu_fetch_sys_files(dev);
if (ret)
    dev_warn(&vdev->dev, "virtio-gpu-nv: GET_SYS_FILES failed: %d\n", ret);

/* Create /sys/devices/system/node/node0 with real host cpumap/cpulist */
nvgpu_numa_init(dev); /* non-fatal */

/* Create /dev/dri/renderD128 etc. with host major:minor */
nvgpu_dri_init(dev); /* non-fatal */

dev_info(&vdev->dev, "virtio-gpu-nv: %u GPU(s), driver %s\n", dev->num_gpus,
    dev->driver_version);
return 0;

err_uvm_region:
unregister_chrdev_region(dev->uvm_devno, 2);
```

```
err_uvm_cdev:
err_ctl_cdev:
    cdev_del(&dev->cdev_ctl);
    device_destroy(nvgpu_class, MKDEV(NV_MAJOR, NV_CTL_MINOR));
err_ctl_region:
    unregister_chrdev_region(MKDEV(NV_MAJOR, NV_CTL_MINOR), 1);
err_gpu_cdevs:
    for (i = i - 1; i >= 0; i--) {
        cdev_del(&dev->cdev_gpu[i]);
        device_destroy(nvgpu_class, MKDEV(NV_MAJOR, i));
    }
    unregister_chrdev_region(MKDEV(NV_MAJOR, 0), dev->num_gpus);
err_class:
    class_destroy(nvgpu_class);
    nvgpu_class = NULL;
    return ret;
}

static void nvgpu_remove(struct virtio_device *vdev) {
    struct nvgpu_device *dev = vdev->priv;
    int i;

    vdev->config->reset(vdev);

    nvgpu_dri_cleanup(dev);
    nvgpu_numa_cleanup(dev);
    g_nvgpu_dev = NULL;

    for (i = 0; i < (int)dev->num_gpus; i++) {
        device_destroy(nvgpu_class, MKDEV(NV_MAJOR, i));
        cdev_del(&dev->cdev_gpu[i]);
    }
    unregister_chrdev_region(MKDEV(NV_MAJOR, 0), dev->num_gpus);

    device_destroy(nvgpu_class, MKDEV(NV_MAJOR, NV_CTL_MINOR));
    cdev_del(&dev->cdev_ctl);
    unregister_chrdev_region(MKDEV(NV_MAJOR, NV_CTL_MINOR), 1);

    device_destroy(nvgpu_class, dev->uvm_devno);
    device_destroy(nvgpu_class, MKDEV(MAJOR(dev->uvm_devno), 1));
    cdev_del(&dev->cdev_uvm);
    unregister_chrdev_region(dev->uvm_devno, 2);

    if (nvgpu_class) {
        class_destroy(nvgpu_class);
        nvgpu_class = NULL;
    }

    vdev->config->del_vqs(vdev);

    remove_proc_subtree("driver/nvidia", NULL);
}

/* â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200 Modu
le boilerplate â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â\224\200â
\224\200 */

static struct virtio_device_id id_table[] = {
    {VIRTIO_ID_GPU_NV, VIRTIO_DEV_ANY_ID},
    {0},
};
MODULE_DEVICE_TABLE(virtio, id_table);

static unsigned int features[] = {
    VIRTIO_F_VERSION_1,
    VIRTIO_GPU_NV_F_UVM,
    VIRTIO_GPU_NV_F_ENCODE,
    VIRTIO_GPU_NV_F_GRAPHICS,
};
```

```
static struct virtio_driver nvgpu_driver = {
    .driver.name = "virtio-gpu-nv",
    .driver.owner = THIS_MODULE,
    .id_table = id_table,
    .feature_table = features,
    .feature_table_size = ARRAY_SIZE(features),
    .probe = nvgpu_probe,
    .remove = nvgpu_remove,
};

module_virtio_driver(nvgpu_driver);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("libkrun contributors");
MODULE_DESCRIPTION("virtio-gpu-nv: NVIDIA GPU sharing for VMs via ioctl proxy");
```